1) Can we use multiple catch blocks in Java?

- Yes, we can use multiple catch blocks for different exceptions.
- Each catch block handles a specific type of exception.
- We must write child exception first, then parent exception.

Example:

```
public class Test {
   public static void main(String[] args) {

      try {
         int a = 10 / 0;     // this causes ArithmeticException

      } catch (ArithmeticException e) {
         System.out.println("Arithmetic Exception handled");   // handles divide by
                                                                  zero error

      } catch (Exception e) {
         System.out.println("General Exception handled");     // handles any other
                                                                  exception

      }

   }
}
```

Output:
Arithmetic Exception handled

TreeofCoders

2) Can we handle multiple exceptions in single catch block?

- Yes, We can handle multiple exceptions using single catch block with | (pipe).
- This feature is called multi-catch, introduced in Java 7.
- It reduces code duplication and makes code clean.

Example:

```
public class Test {
   public static void main(String[] args) {

      try {
         int a = 10 / 0;    // may cause ArithmeticException

         int arr[] = new int[2];
         arr[5] = 10;      // may cause ArrayIndexOutOfBoundsException
      }
      catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
         System.out.println("Exceptions handled");     // handles both exceptions
                                                          in single catch block
      }

   }
}
```

Output:
Exceptions handled

TreeofCoders

3) Is finally block will always execute if any exception occurs?

- Yes, finally block will execute in most cases.
- It runs whether exception occurs or not.
- But it will NOT run if JVM stops (like System.exit())

Example:

```
public class Test {
   public static void main(String[] args) {

      try {
         int a = 10 / 0;     // this causes ArithmeticException
      }
      catch (Exception e) {
         System.out.println("Exception handled");     // catch block runs
      }
      finally {
         System.out.println("Finally block executed");    // this always runs
                                                              (normal case)
      }

   }
}
```

Output:
Exception handled
Finally block executed

TreeofCoders

4) Is it possible to terminate finally block?

- Yes, it is possible to stop finally block execution.
- If we use System.exit(), JVM will stop immediately.
- So finally block will not execute.

Example:

```
public class Test {
   public static void main(String[] args) {

      try {
         System.out.println("Try block");

         System.exit(0);     // JVM stops here → program ends immediately
      }

      finally {
         System.out.println("Finally block");     // this will NOT execute
      }

   }
}
```

Output: Try block

TreeofCoders

FOR MORE, MOST ASKED
JAVA INTERVIEW QUES-ANS IN ONE PDF

DOWNLOAD PDF → ***Click here to download PDF***

ONE SIMPLE ORGANIZED GUIDE

8 DAYS QUICK PREPARATION

Java

DAYS	SUBJECT	
DAY 1	Core Java (Topic wise QA + Coding)	Startups & Big MNCs
DAY 2	Core Java (Most Important QA + Coding)	
DAY 3	DSA Coding Practice	
DAY 4	Advanced Java (Topic wise QA + Coding)	
DAY 5	Advanced Java (Most Important QA + Coding)	
DAY 6	TCS + Infosys + Wipro + HCL	
DAY 7	Cognizant + Capgemini + Accenture	
DAY 8	HR Round QA	